

California State University Long Beach
Computer Engineering and Computer Science Department
CECS 440 – Computer Architecture
Lab 9 – MIPS Instruction Fetch Stage

Objectives:

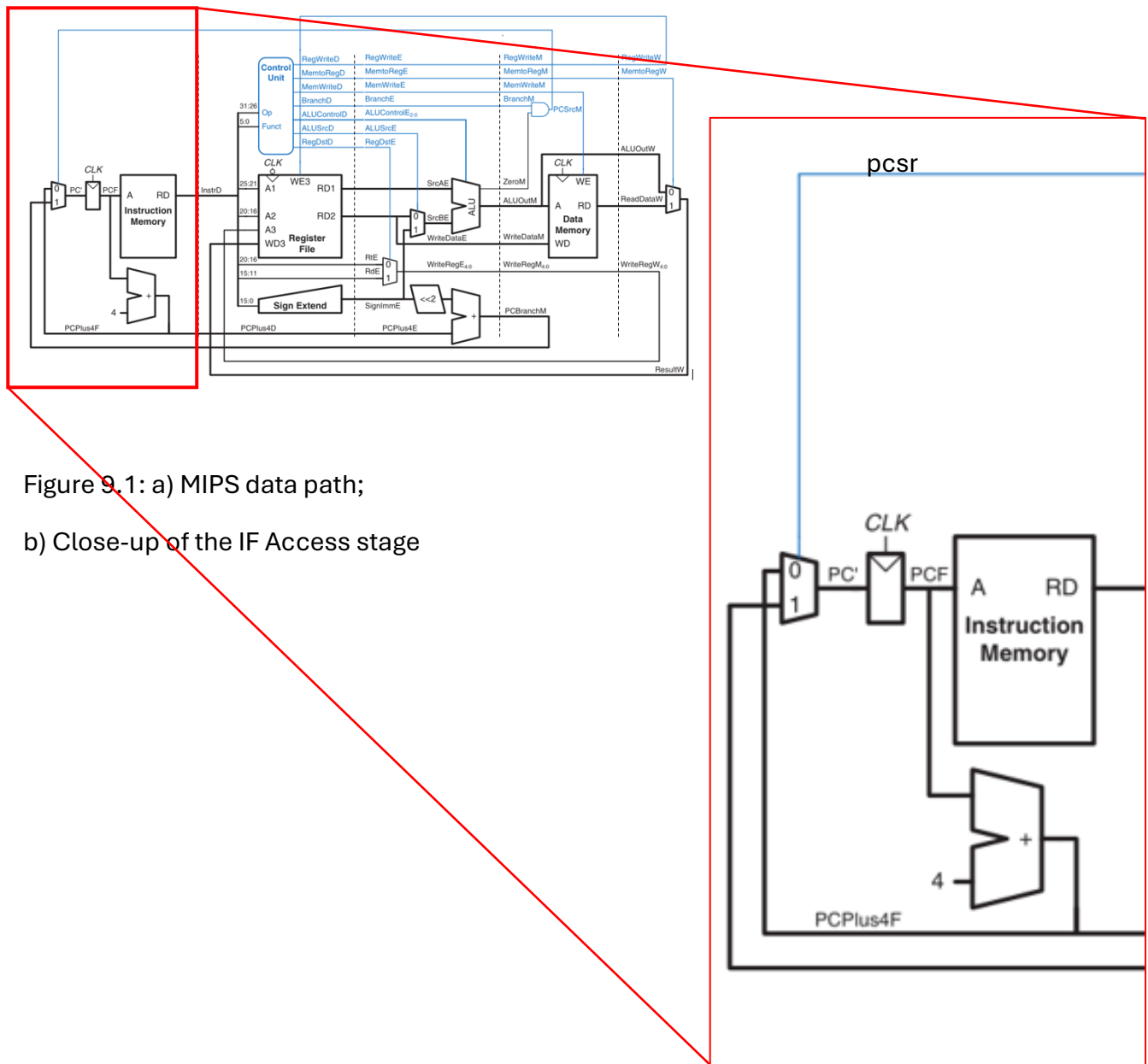
- Implement the **Instruction Fetch (IF) stage** of a single-cycle MIPS processor in Verilog (PC register, PC+4 adder, IMEM interface, next-PC MUX).
- Use control signal **PCSrc** to manage next-PC selection (PC+4 vs. branch target) and ensure correct PC update on clock/reset.
- Connect and verify data paths for **PC**, **PCPlus4**, **BranchTarget**, and **instr** between IF and downstream stages.
- Fetch the 32-bit instruction (**instr**) from **IMEM** using the current **PC**, ensuring proper word alignment.
- Simulate and confirm correct instruction fetch and PC sequencing (taken and not-taken paths) using Xilinx Vivado behavioral simulation with a Verilog testbench.

Background:

In the single-cycle MIPS processor, instruction execution occurs across five stages. The **Instruction Fetch (IF) stage** is responsible for retrieving the machine code of the instruction from **Instruction Memory (IMEM)** and updating the **Program Counter (PC)**. During this stage, the current instruction is fetched, the PC value is incremented and then updated according to the next PC logic.

The key signals used in the IF stage are described below:

- **pcsrc**: Controls how the multiplexer (MUX) updates the PC. When **pcsrc** = 0, the PC increments normally to the next instruction. When **pcsrc** = 1, a branch is taken, and the PC is loaded with the branch target address.
- **instr**: The 32-bit machine code of the instruction fetched from **IMEM**.



Steps:

1. Get the mux2.v file from a previous lab.
2. A configurable size register is needed. Use reg.v with Verilog module definition given below:

```
1 module register #(parameter WIDTH = 8)
2     (input clk, reset,
3       input [WIDTH-1:0] d,
4       output reg [WIDTH-1:0] q
5     );
6
7     always @(posedge clk, posedge reset)
8         if (reset) q = 0;
9         else      q = d;
10 endmodule
```

Figure 9.2: Register module

3. Create a 32-bit adder in a file named **adder32.v** which is defined below:

```
1 module adder32 (
2     input [31:0] a, b,
3     output [31:0] y
4 );
5
6     assign y = a + b;
7 endmodule
```

Figure 9.3: 32-bit adder module

4. Create Instruction Memory in a file named **imem.v** and name the module **imem**:

```
1 module imem(  
2     input [5:0] a,  
3     output [31:0] rd  
4 );  
5  
6     reg [31:0] RAM[63:0];  
7     assign rd = ;  
8 endmodule
```

Figure 9.4: Instruction memory module

5. Use the block diagram on page 1 and complete the module skeleton below to create the IF stage:

```
1 module IF_Stage (  
2     input clk, psrc, reset,  
3     input [31:0] pcbranch,  
4     output [31:0] instr,  
5     output [31:0] pcplus4  
6 );  
7  
8     wire [31:0] pc, pcnext;  
9     assign pc[31:8] = 0;  
10  
11     // Complete this  
12     register pcreg(      );  
13  
14     adder32 pcadd(  
15         .a( {24'h0, pc[7:0]} ),  
16         .b( 32'h4 ),  
17         // Complete this  
18     );  
19  
20     // Complete this  
21     mux2 #(32) pcbrmux(      );  
22  
23     // Complete this  
24     imem imem(      );  
25  
26 endmodule
```

Figure 9.5: Skeleton of the code of IF Stage module

6. Test your design to ensure it works correctly using the Verilog test fixture code below:

```

1 module t_IF;
2     reg clk, pcsrc, reset;
3     reg [31:0] pcbranch;
4     wire [31:0] instr, pcplus4;
5
6     IF_Stage dut(clk, pcsrc, reset, pcbranch, instr, pcplus4);
7
8     integer i;
9     always #5 clk = ~clk;
10    initial begin
11        clk = 1;
12        reset = 1;
13        #1
14        reset = 0;
15        for ( i = 0; i < 64; i = i + 1)
16            dut.imem.RAM[i] = $random;
17        for ( i = 0; i < 10; i = i + 1)
18            testcase;
19        #1 $finish;
20    end
21
22    task testcase;
23        begin
24            @(negedge clk) {clk, pcsrc, reset, pcbranch} = $random;
25            reset = 0;
26
27            @(posedge clk) $display("Test Case %d", i);
28            #1 $display("pcsrc = %b\tpcbranch = %h", pcsrc, pcbranch);
29            $display("pc = %h\tinstr = %h", dut.pc, instr);
30            $display("pcnext=%h\t reset=%h", dut.pcnext, dut.reset);
31        end
32    endtask
33
34 endmodule

```

Figure 9.6: Testbench for the IF_Stage module

Correct test results are shown below:		Test Case		5
Test Case	0	pcsrc = 0	pcbranch = 0aaa4b15	
pcsrc = 0	pcbranch = 42f24185	pc = 00000082	instr = c03b2280	
pc = 00000004	instr = c0895e81	pcnext=00000086	reset=0	
pcnext=00000008	reset=0	Test Case	6	
Test Case	1	pcsrc = 0	pcbranch = 78d99bf1	
pcsrc = 0	pcbranch = 27f2554f	pc = 00000086	instr = 10642120	
pc = 00000008	instr = 8484d609	pcnext=0000008a	reset=0	
pcnext=0000000c	reset=0	Test Case	7	
Test Case	2	pcsrc = 0	pcbranch = 6c9c4bd9	
pcsrc = 1	pcbranch = 9dcc603b	pc = 0000008a	instr = 557845aa	
pc = 0000003b	instr = 7cfde9f9	pcnext=0000008e	reset=0	
pcnext=9dcc603b	reset=0	Test Case	8	
Test Case	3	pcsrc = 0	pcbranch = 31230762	
pcsrc = 0	pcbranch = 1d06333a	pc = 0000008e	instr = cecccc9d	
pc = 0000003f	instr = e33724c6	pcnext=00000092	reset=0	
pcnext=00000043	reset=0	Test Case	9	
Test Case	4	pcsrc = 0	pcbranch = 2635fb4c	
pcsrc = 1	pcbranch = bf23327e	pc = 00000092	instr = cb203e96	
pc = 0000007e	instr = 0573870a	pcnext=00000096	reset=0	
pcnext=bf23327e	reset=0			

Figure 9.7: Expected output for the IF_Stage module

WHAT TO SUBMIT

Once you have verified that your project is functioning correctly, copy the contents of **design.v**, **mux.v**, **reg.v**, **addr32.v**, **imem.v** and **design_tb.v** into a single text file named **lab9.txt**. Submit **this file** along with your **lab report** on Canvas.

Write your report using the **Lab Report Template** available on Canvas. Be sure to include:

- Screenshots of your code
- Console output
- One waveform screenshot showing a few selected test cases

Note: Keep all lab files, as they will be needed for future labs.